



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DIPARTIMENTO DI
INFORMATICA - SCIENZA E INGEGNERIA

EREDITARIETÀ

COSIMO LANEVE

`cosimo.laneve@unibo.it`

CORSO 00819 – PROGRAMMAZIONE

ARGOMENTI (SAVITCH CAPITOLO 15.1)

1. ereditarietà
2. sottotipaggio
3. costruttori ed ereditarietà

EREDITARIETÀ

spesso non si vuole definire una classe a partire dal nulla, ma vogliamo **definirla a partire da** un'altra classe

- * uno `studente` è anche una `persona`
- * se abbiamo già definito una classe `persona` vorremmo riusare il codice già scritto

EREDITARIETÀ – ESEMPIO

- * uno `studente` è una **sottoclasse** di `persona`
 - ha **tutti i campi** di `persona`
 - ha **tutti i metodi** di `persona`, eventualmente modificati (tranne il costruttore)
 - può avere altri metodi e altri campi
- * come `persona`, uno `studente` ha `nome`, `cognome`, `indirizzo`, `telefono`, ...
- * in aggiunta uno `studente` ha una lista di esami fatti e una media voti, che una `persona` generica non ha

PERCHÈ L'EREDITARIETÀ?

perchè si desidera riusare il codice già scritto

- * non si riscrivono tutti i campi e i metodi di una classe
- * si scrivono solamente quelli **nuovi** e/o **modificati**

il riuso del codice aiuta a risparmiare

- * **tempo di scrittura**: meno codice da scrivere
- * **tempo di debugging**: meno codice = meno errori e quindi meno debugging
- * **tempo di modifica**: meno codice da modificare
- * **denaro**: se pago qualcuno per scrivere il codice

**senza ereditarietà i linguaggi orientati agli oggetti
avrebbero avuto meno successo**

SINTASSI

questo è l'access-level: definisce il tipo della derivazione (in questo corso sarà sempre `public`)



```
class nomeFiglio: public nomePadre{  
    campi nuovi e metodi nuovi o modificati  
}
```

- * **nomeFiglio** è il nome della nuova classe, **nomePadre** il nome della classe da cui si eredita
- * la nuova classe **ha tutti i campi della classe vecchia**, più quelli nuovi
- * la nuova classe **ha tutti i metodi della classe vecchia** (tranne il costruttore), eventualmente modificati, più quelli nuovi
- * il prototipo del metodo modificato deve essere uguale a quello del corrispondente metodo nel padre

TERMINOLOGIA

- * studente si dice classe derivata, o **sottoclasse**, o classe figlio di persona
- * viceversa, persona si dice classe base, o classe padre, o **superclasse** di studente
- * la modifica di un metodo della classe padre nella sottoclasse si chiama **overriding**
- * per accedere ad un metodo della classe padre:
`Nome_ClassePadre::metodo(e1, ..., en)`

ESEMPIO

```
class Persona {
protected:
    char nome[50];
    int eta;
public:
    Persona(char n[]="", int e=0){
        strcpy(nome,n); eta=e;
    }
    void presentati(){
        cout << "Sono " << nome << ", ho " << eta << " anni" ;
    }
};
```

non c'è riuso di codice!
abbiamo ridefinito tutto ...

```
class Studiante: public Persona {
protected:
    double media;
public:
    Studiante(char n[], int e, double m){
        strcpy(nome,n); eta=e; media=m;
    }
    void presentati(){
        cout << "Sono " << nome << ", ho " << eta << " anni e media " << media ;
    }
};
```

← Studiante è sottoclasse di Persona

ESEMPIO

riuso del codice per i costruttori?
... per il momento non c'è ...

```
class Persona {
protected:
    char nome[50];
    int eta;
public:
    Persona(char n[],int e=0){
        strcpy(nome,n); eta=e;
    }
    void presentati(){
        cout << "Sono " << nome << ", ho " << eta << " anni" ;
    }
};

class Studente: public Persona {
protected:
    double media;
public:
    Studente(char n[], int e, double m){
        strcpy(nome,n); eta=e; media=m;
    }
    void presentati(){
        Persona::presentati() ;
        cout << " e media " << media ;
    }
};
```

riuso di codice!

... l'esempio è molto semplice ...

ALTRO ESEMPIO

```
class Rectangle{
protected:
    double base ;
    double altezza ;
public:
    double area(){ return(base*altezza) ;
    }
    double perimeter(){ return(2*base + 2*altezza) ;
    }
    Rectangle(int m = 0 , int n = 0){ base = abs(m); altezza = abs(n);
    }
} ;

class Triangle_Rect : public Rectangle {
public:
    double area(){ return(Rectangle::area()/2 );
    }
    double perimeter(){
        return(base + altezza + sqrt(base*base + altezza*altezza));
    }
    Triangle_Rect(int m,int n){ base = abs(m); altezza = abs(n); }
} ;
```

c'è riuso di codice!
... non per i costruttori ...

RECAP: RICHIAMARE I METODI RIDEFINITI

- * quando un metodo è ridefinito nella sottoclasse (**overriding**), si può ancora accedere all'omonimo metodo nella superclasse
- * lo si richiama con `Nome_ClassePadre::metodo`

```
class Persona{
    ...
    void presentati(){
        cout << "Sono " << nome << " e ho " << eta << " anni" ;
    }
    ...
};
class Studente: public Persona{
    ...
    void presentati(){
        Persona::presentati();
        cout << " e media " << media ;
    }
    ...
};
```

I COSTRUTTORI NON SONO EREDITATI

prima di invocare il costruttore di una sottoclasse viene invocato il costruttore di default della superclasse

- * in questo modo il costruttore della sottoclasse può assumere che i campi della superclasse siano già stati inizializzati
- * se il costruttore di default della superclasse **non esiste si ha un errore**

ESEMPI SUI COSTRUTTORI

```
class A {  
protected:  
    int uno ;  
public:  
    A(int n = 0){  
        uno = n ;  
        cout << "costruttore classe di base A" << endl ;  
    }  
};
```

se si omette il valore di default
■ ERRORE

```
class B : public A{  
protected:  
    string due ;  
public:  
    B(string x = "Undefined"){  
        due = x ;  
        cout << "costruttore classe derivata B" << endl ;  
    }  
};  
  
int main() {  
    B z = B("ciao") ;  
    return 0;  
}
```

console

costruttore classe di base A
costruttore classe derivata B

ESEMPI SUI COSTRUTTORI

```
class A{
protected:
    int uno ;
public:
    A(){
        uno = 0 ;
        cout << "costruttore di default classe di base A" << endl ;
    }
    A(int n){
        uno = n ;
        cout << "costruttore classe di base A" << endl ;
    }
};

class B: public A {
protected:
    string due ;
public:
    B(string x = "Undefined"){
        due = x ;
        cout << "costruttore classe derivata B" << endl ;
    }
};

int main() {
    B z = B("ciao") ;
    return 0;
}
```

ATTENZIONE

■ **overloading**

se si scrive il valore di default

■ **ERRORE**

console

costruttore di default classe di base A
costruttore classe derivata B

ANCORA SUI COSTRUTTORI

una sottoclasse può dichiarare quale costruttore della classe base vuole usare

```
class Persona{
    ...
    Persona(char n[], int e){ strcpy(nome,n); eta=e; }
    ...
};

class Studente: public Persona{
    ...
    Studente(char n[], int e, double m): Persona(n,e){
        media = m ;
    }
    ...
};
```

una sottoclasse è un SOTTOTIPO della sua superclasse

- * quindi è possibile usare un oggetto della sottoclasse in qualunque contesto serva un oggetto della superclasse
 - un oggetto della sottoclasse **si può assegnare** ad una variabile della superclasse
 - un oggetto della sottoclasse **si può passare come parametro** ad una funzione che si aspetta un oggetto della superclasse
 - un oggetto della sottoclasse **si può ritornare come valore di ritorno** se il tipo di tale valore deve essere una superclasse
 - un oggetto della sottoclasse **si può farlo puntare** da un puntatore alla superclasse
- * Il contrario non è vero

SOTTOTIPAGGIO: ESEMPIO

```
class Rectangle{
protected:
    double base ;
    double altezza ;
public:
    double area(){ return(base*altezza) ; }
    double perimeter(){ return(2*base + 2*altezza) ; }
    Rectangle(int m =0 , int n=0){ base = abs(m); altezza = abs(n); }
} ;
```

static dispatch in C++

```
class Triangle_Rect : public Rectangle {
public:
    double area(){ return(Rectangle::area()/2) ; }
    double perimeter(){
        return(base + altezza + sqrt(base*base + altezza*altezza));}
    Triangle_Rect(int m,int n): Rectangle(m, n){}
} ;
```

classe Triangle_Rect
sottotipo di Rectangle

```
int main() {
    Triangle_Rect t = Triangle_Rect(3,4) ;
    Rectangle r = Rectangle(2,3) ;
    r = t ;
    cout << t.perimeter() << '\t' << r.perimeter() ;
}
```

r = t è possibile perchè
Triangle_Rect è sottotipo di
Rectangle

output: 12 14

SOTTOTIPAGGIO: ALTRO ESEMPIO

```
class Rectangle{
protected:
    double base ;
    double altezza ;
public:
    double area(){ return(base*altezza) ; }
    double perimeter(){ return(2*base + 2*altezza) ; }
    Rectangle(int m =0 , int n=0){ base = abs(m); altezza = abs(n); }
} ;
```

static dispatch in C++

```
class triangle_Rect : public Rectangle {
public:
    double area(){ return((base*altezza)/2) ;}
    double perimeter(){ return(base + altezza + sqrt(base*base + altezza*altezza)); }
    Triangle_Rect(int m,int n){ base = abs(m); altezza = abs(n); }
} ;
```

classe Triangle_Rect
sottotipo di Rectangle

```
Rectangle give_me_a_rect(Rectangle x){
    double p = x.perimeter() ;
    Triangle_Rect z = Triangle_Rect(p/2,p/2) ;
    return(z) ;
}
```

ritorna uno triangle che è
tipato come Rectangle

```
int main() {
    Triangle_Rect q = Triangle_Rect(3,4) ;
    Rectangle r = give_me_a_rect(q) ;
    cout << r.perimeter() ;
}
```

give_me_a_rect è invocata con
un Triangle_Rect

output: 28

GERARCHIA DI CLASSI

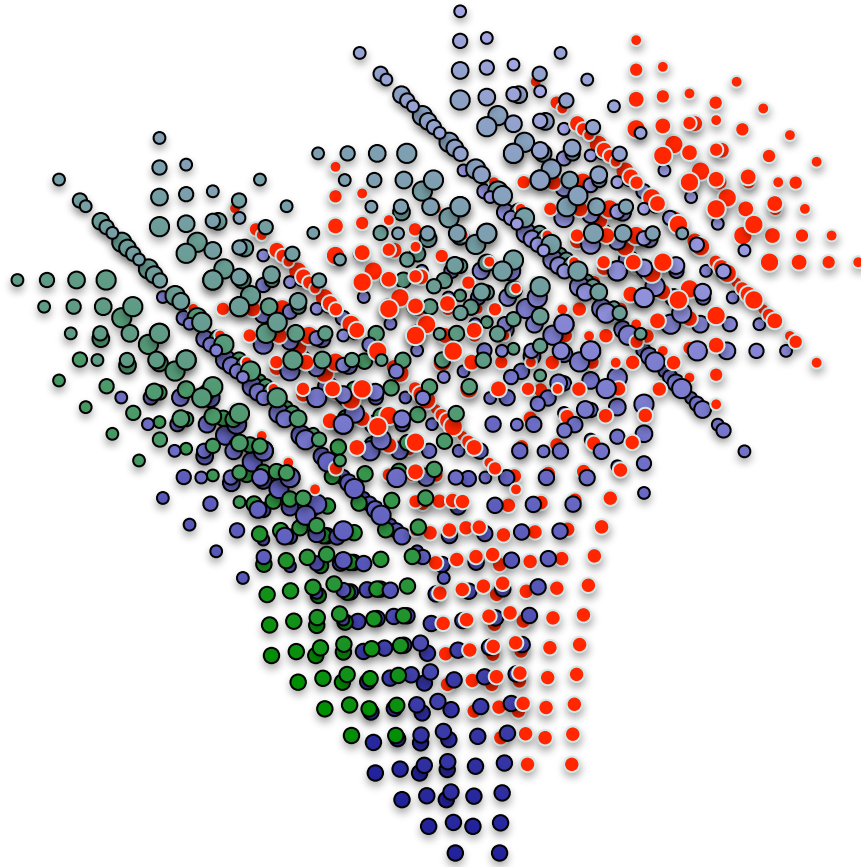
- * una sottoclasse può essere a sua volta la superclasse di un'altra classe
- * `Studiante` è una sottoclasse di `Persona`, e una superclasse di `Laureando`
- * un `Laureando` ha i metodi e campi di `Persona`, quelli di `Studiante` più altri, ad esempio `relatore` e `titoloTesi`
- * in questo modo è possibile definire anche gerarchie complicate

ESERCIZI

1. definire una classe `contoBanca` con un `saldo` e metodi `versa` e `preleva`. Definire una sottoclasse `contoInteressi` con un metodo aggiuntivo che aumenta il saldo del 2%.
2. definire una classe `persona` con campi opportuni, un costruttore e metodo `presentati`. Definire una sottoclasse `impiegato` con, in più, i campi `reparto` e `stipendio`, un metodo `presentati` opportunamente modificato e un metodo `aumento` che aumenta lo stipendio del 5%.

ESERCIZI

1. Definire una classe **Città** con campi **nome** e **numeroAbitanti**, con un opportuno costruttore e metodi **descrizione** (che stampa le informazioni sulla città) e **cambiaAbitanti**, che cambia il numero di abitanti.
2. Definire una classe **Capoluogo** che è una **Città** con in più l'informazione sulla **Regione** di cui è capoluogo.
3. Definire una classe **Capitale** che è un **Capoluogo** con in più l'informazione sulla **nazione** di cui è capitale.
4. Creare gli oggetti per **cesenatico**, **bologna**, **torino** e **roma** e stamparne le informazioni.



FINE